

Cross-Lingual Question Answering with Custom Transformer Models

Team Number 9

BT2024249 Parikshith Aithal K V

BT2024094 Nishanth Karamsetty

BT2024246 Adithya Kommuri

BT2024243 Shivansh Shah

GitHub Repository:

[View GitHub Repository](#)

Google Drive (Models):

[Open Drive Folder](#)

Note: Large model files are hosted on Google Drive as they exceed GitHub size limits.

NLP Project

April 2026

Abstract

This report presents the complete architecture, implementation details, training methodology, and experimental evaluation of an end-to-end Natural Language Processing system for **cross-lingual question answering between Kannada and English**. The system comprises four major components, all built from scratch without any pre-trained model weights: (1) a 110M-parameter BERT-style encoder pre-trained via Masked Language Modeling (MLM) on English Wikipedia for 100,000 steps; (2) an extractive reading-comprehension model fine-tuned on SQuAD v1.1; (3) bidirectional English–Kannada neural machine translation models implemented both with PyTorch `nn.Transformer` modules and with a fully hand-coded Transformer from scratch; and (4) a Streamlit-based web front-end that orchestrates the full pipeline. We report detailed training curves, throughput benchmarks, and a **comparative evaluation** of both translation backends (`nn.Transformer` vs. Scratch), demonstrating that the from-scratch model achieves competitive BLEU (13.15 vs. 14.59) and **superior F1** (0.367 vs. 0.356) on a single consumer-grade NVIDIA RTX 4060 GPU.

Keywords: Transformer, BERT, Masked Language Modeling, Extractive QA, SQuAD, Neural Machine Translation, Kannada, SentencePiece, Cross-Lingual NLP

Contents

1	Introduction	4
1.1	Data Sources	4
1.2	Report Structure	4
2	System Architecture Overview	5
2.1	Training Pipeline	5
3	Stage 1: Masked Language Model Pre-training	5
3.1	What the Code Does	5
3.2	Model Architecture	6
3.2.1	Embedding Layer	6
3.2.2	Multi-Head Self-Attention	7
3.2.3	Transformer Block with Pre-Norm Residuals	8
3.2.4	MLM Head with Weight Tying	8
3.3	Data Pipeline: Streaming Wikipedia	8
3.4	Training Configuration and Results	9
3.5	MLM Test Cases	10
4	Stage 2: Extractive QA Fine-tuning on SQuAD	11
4.1	What the Code Does	11
4.2	Reader Model Architecture	12
4.3	Position Embedding Resizing ($128 \rightarrow 512$)	12
4.4	Fine-tuning Configuration and Results	13
4.5	QA Test Cases	13
4.6	SQuAD Evaluation (F1 / EM)	14
5	Stage 3a: Translation with nn.Transformer	15
5.1	What the Code Does	15
5.2	Model Architecture	15
5.3	Training Data Pipeline	16
6	Stage 3b: Translation from Scratch (No nn.Transformer)	17
6.1	What the Code Does	17
6.2	Hand-Written Layer Normalisation	18
6.3	Hand-Written Sinusoidal Positional Encoding	18
6.4	Hand-Written Multi-Head Attention	18
6.5	Scratch Transformer Assembly	19
6.6	Guarded Greedy Decoding	20
6.7	Translation Training Configuration	22
7	Comparative Evaluation: nn.Transformer vs. Scratch	22
7.1	BLEU Evaluation Methodology	22
7.2	Results	23
7.3	Analysis	23
8	End-to-End Pipeline and Deployment	24
8.1	Pipeline Orchestration	24

8.2	Streamlit Web Application	25
8.3	Pipeline Test Cases	26
9	Model Checkpoint Registry	26
10	Design Decisions and Lessons Learned	26
11	Future Work	27
12	Conclusion	27
A	Command Reference	27
B	Full Model Configurations	28

1 Introduction

Natural Language Understanding in low-resource languages remains one of the most challenging open problems in modern NLP. While English has benefited enormously from large-scale pre-trained models (BERT, GPT, T5), many languages spoken by tens of millions of people still lack comparable tooling. Kannada, a Dravidian language spoken by approximately 45 million people in southern India, is one such language.

This project addresses the gap by constructing a complete **cross-lingual Question Answering (QA) pipeline** that enables a user to:

1. Pose a question in **Kannada**.
2. Translate the question to **English** using a custom neural machine translation (NMT) model.
3. Run an **extractive QA reader** over an English context passage to locate the answer span.
4. Translate the extracted English answer **back to Kannada**.

A distinguishing feature of this project is that *all* neural components are trained from scratch. No external pre-trained weights (e.g., from Hugging Face Model Hub) are used.

1.1 Data Sources

All training data comes from publicly available sources:

Table 1: Training data sources used across all stages

Stage	Dataset	Source	Size
MLM Pre-training	English Wikipedia <i>Config: 20231101.en, streamed via HuggingFace datasets</i>	wikimedia/wikipedia	~6M articles
QA Fine-tuning	SQuAD v1.1 <i>Format: JSON with context paragraphs and question-answer pairs</i>	Stanford NLP	87,599 train / 10,570 dev
Translation	AI4Bharat Samanantar <i>Config: kn (Kannada), parallel EN-KN sentence pairs</i>	ai4bharat/samanantar	600,000 pairs (capped)

1.2 Report Structure

This report follows the natural architectural order of the system:

1. **Section 2:** System architecture overview
2. **Section 3:** Stage 1 — MLM pre-training (encoder)
3. **Section 4:** Stage 2 — SQuAD QA fine-tuning
4. **Section 5:** Stage 3a — Translation with `nn.Transformer`
5. **Section 6:** Stage 3b — Translation from scratch

6. **Section 7:** Comparative evaluation of both backends

7. **Section 8:** End-to-end pipeline and deployment

2 System Architecture Overview

Figure 1 illustrates the high-level data flow through the deployed cross-lingual QA system.

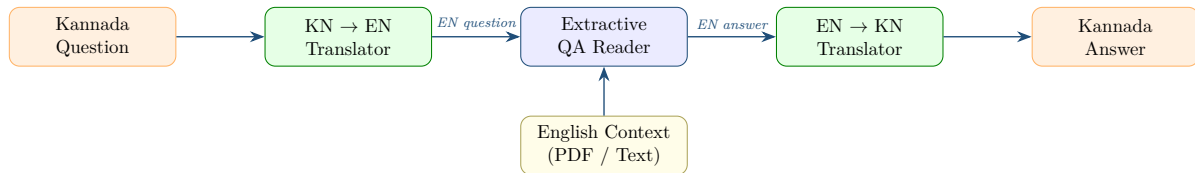


Figure 1: End-to-end cross-lingual QA pipeline architecture.

2.1 Training Pipeline

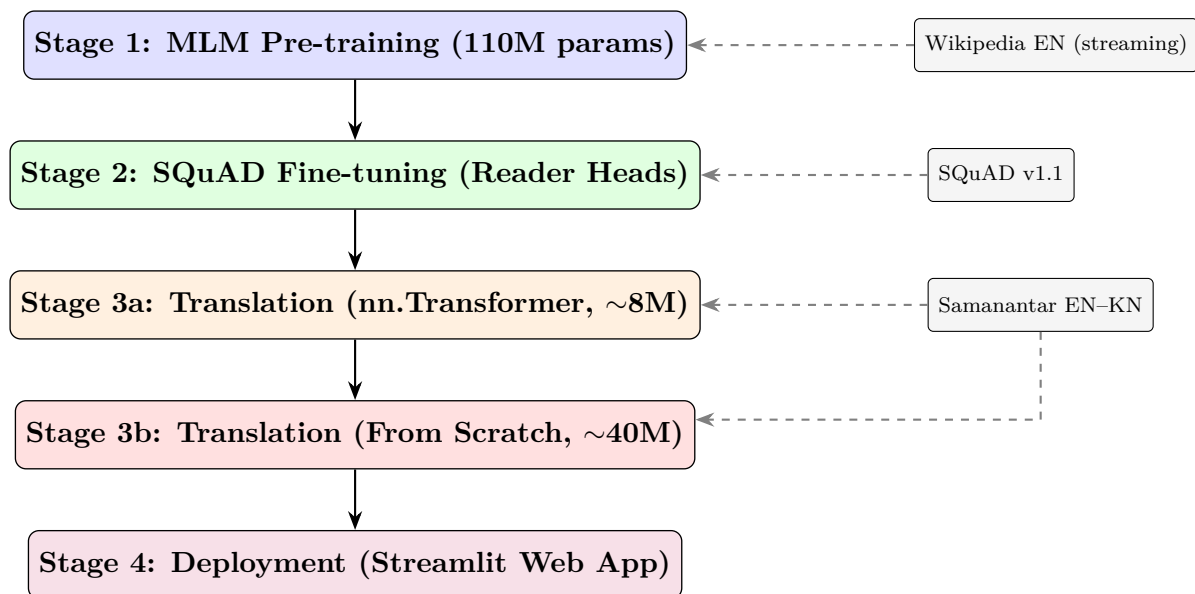


Figure 2: Four-stage training pipeline with data sources.

3 Stage 1: Masked Language Model Pre-training

The first stage builds a general-purpose English language encoder via Masked Language Modeling (MLM) on English Wikipedia, following the BERT methodology [?].

3.1 What the Code Does

The MLM system has four components:

- **config.py:** Defines the `ModelConfig` dataclass holding all architecture hyperparameters.

- **model.py**: Implements the full BERT encoder (`BertEmbeddings`, `MultiHeadSelfAttention`, `TransformerBlock`, `BertEncoder`) and the `MLMHead` with weight tying.
- **data.py**: Streaming Wikipedia data pipeline that tokenises articles on-the-fly and applies the 15% MLM masking strategy (80% [MASK], 10% random, 10% unchanged).
- **train.py**: Training loop with mixed-precision (FP16), gradient accumulation, linear LR warmup/decay, `torch.compile`, and periodic checkpointing.

3.2 Model Architecture

The encoder uses 12 transformer layers with **pre-norm residual connections** (Layer Norm applied *before* each sub-layer).

```

1  from dataclasses import dataclass
2
3  @dataclass
4  class ModelConfig:
5      vocab_size: int                # 30,522 (bert-base-
6          uncased)                  # absolute position
7      max_position_embeddings: int = 512  # slots
8      type_vocab_size: int = 2        # segment A / B
9      hidden_size: int = 768          # embedding dimension
10     num_hidden_layers: int = 12      # transformer blocks
11     num_attention_heads: int = 12     # heads per layer
12     intermediate_size: int = 3072    # FFN hidden dim (4x)
13     hidden_dropout_prob: float = 0.1
14     attention_probs_dropout_prob: float = 0.1
15     layer_norm_eps: float = 1e-12

```

Listing 1: Model configuration (`config.py`) — defines the BERT-base-scale architecture

With vocabulary $V = 30,522$, this yields **~110M parameters**.

3.2.1 Embedding Layer

Sums word embeddings, learned absolute position embeddings, and segment embeddings, then applies LayerNorm and dropout:

```

1  class BertEmbeddings(nn.Module):
2      def __init__(self, config: ModelConfig):
3          super().__init__()
4          self.word_embeddings = nn.Embedding(
5              config.vocab_size, config.hidden_size)
6          self.position_embeddings = nn.Embedding(
7              config.max_position_embeddings, config.hidden_size)
8          self.token_type_embeddings = nn.Embedding(
9              config.type_vocab_size, config.hidden_size)
10         self.layer_norm = nn.LayerNorm(
11             config.hidden_size, eps=config.layer_norm_eps)
12         self.dropout = nn.Dropout(config.hidden_dropout_prob)
13
14         def forward(self, input_ids, token_type_ids):
15             B, S = input_ids.shape

```

```

16     pos_ids = torch.arange(S, device=input_ids.device) \
17         .unsqueeze(0).expand(B, S)
18     emb = (self.word_embeddings(input_ids)
19           + self.position_embeddings(pos_ids)
20           + self.token_type_embeddings(token_type_ids))
21     return self.dropout(self.layer_norm(emb))

```

Listing 2: Embedding layer (model.py) — three embedding types summed together

3.2.2 Multi-Head Self-Attention

Uses PyTorch’s `F.scaled_dot_product_attention` which dispatches to FlashAttention-2 on compatible GPUs for $O(N)$ memory:

```

1  class MultiHeadSelfAttention(nn.Module):
2      def __init__(self, config: ModelConfig):
3          super().__init__()
4          self.num_heads = config.num_attention_heads
5          self.head_dim = config.hidden_size // self.num_heads
6          self.q_proj = nn.Linear(config.hidden_size, config.
7                                  hidden_size)
8          self.k_proj = nn.Linear(config.hidden_size, config.
9                                  hidden_size)
10         self.v_proj = nn.Linear(config.hidden_size, config.
11                                 hidden_size)
12         self.out_proj = nn.Linear(config.hidden_size, config.
13                                 hidden_size)
14         self.attn_dropout = nn.Dropout(config.
15                                         attention_probs_dropout_prob)
16         self.proj_dropout = nn.Dropout(config.hidden_dropout_prob)
17
18     def forward(self, hidden_states, attention_mask):
19         B, S, H = hidden_states.shape
20         # Project to Q, K, V and reshape for multi-head
21         q = self.q_proj(hidden_states).view(
22             B, S, self.num_heads, self.head_dim).transpose(1, 2)
23         k = self.k_proj(hidden_states).view(
24             B, S, self.num_heads, self.head_dim).transpose(1, 2)
25         v = self.v_proj(hidden_states).view(
26             B, S, self.num_heads, self.head_dim).transpose(1, 2)
27
28         # Efficient scaled dot-product attention
29         attn_mask = attention_mask[:, None, None, :].bool()
30         ctx = F.scaled_dot_product_attention(
31             q, k, v, attn_mask=attn_mask,
32             dropout_p=self.attn_dropout.p if self.training else 0.0)
33         ctx = ctx.transpose(1, 2).contiguous().view(B, S, H)
34         return self.proj_dropout(self.out_proj(ctx))

```

Listing 3: Multi-head self-attention (model.py) — uses SDPA for automatic FlashAttention dispatch

3.2.3 Transformer Block with Pre-Norm Residuals

Pre-norm applies LayerNorm *before* each sub-layer, improving gradient flow for training stability from scratch:

```

1 class TransformerBlock(nn.Module):
2     def forward(self, x, attention_mask):
3         # Pre-norm: LayerNorm first, then sub-layer, then residual
4         # add
5         x = x + self.attn(self.ln1(x), attention_mask) # attention
6         x = x + self.ffn(self.ln2(x)) # feed-
7         # forward
8     return x

```

Listing 4: Pre-norm transformer block — LN before attention and FFN, skip connections after

3.2.4 MLM Head with Weight Tying

The prediction head shares its output weight matrix with the input word embeddings (weight tying). Since the input and output vocabularies are identical, sharing these weights not only reduces the model size by $\sim 23\text{M}$ parameters, but also accelerates training and acts as a regulariser by forcing the embeddings to be useful both for encoding input tokens and decoding predictions:

```

1 class MLMHead(nn.Module):
2     def __init__(self, config, tied_embedding):
3         super().__init__()
4         self.dense = nn.Linear(config.hidden_size, config.
5                                 hidden_size)
6         self.layer_norm = nn.LayerNorm(config.hidden_size,
7                                         eps=config.layer_norm_eps)
8         self.decoder = nn.Linear(config.hidden_size,
9                                 config.vocab_size, bias=False)
10        self.bias = nn.Parameter(torch.zeros(config.vocab_size))
11        self.decoder.bias = self.bias
12        self.decoder.weight = tied_embedding.weight # weight tying!
13
14    def forward(self, hidden_states):
15        x = F.gelu(self.dense(hidden_states))
16        x = self.layer_norm(x)
17        return self.decoder(x)

```

Listing 5: MLM head (model.py) — projects hidden states to vocabulary with tied weights

3.3 Data Pipeline: Streaming Wikipedia

The data is **streamed** from the Wikimedia dump to avoid a full $\sim 20\text{GB}$ download. Each Wikipedia article is tokenised, split into chunks of `max_len` tokens, and the collator applies the MLM masking strategy:

```

1 class StreamingTextDataset(IterableDataset):
2     def __init__(self, tokenizer, max_len=128, seed=42):

```



```

3         self.tokenizer = tokenizer
4         self.max_len = max_len
5         self.cls_id = tokenizer.cls_token_id
6         self.sep_id = tokenizer.sep_token_id
7         # Stream directly from HuggingFace - no full download
8         self.stream = load_dataset(
9             "wikimedia/wikipedia", "20231101.en",
10            split="train", streaming=True)
11         self.stream = self.stream.shuffle(
12            seed=seed, buffer_size=20000)

```

Listing 6: Streaming Wikipedia loader (data.py) — avoids downloading entire dump

```

1 def mlm_collate(batch, tokenizer, mask_prob=0.15):
2     # For each token selected as a masking candidate:
3     # 80% -> replace with [MASK]
4     # 10% -> replace with random vocabulary token
5     # 10% -> keep original (forces model to also represent
6     unmasked)
7     # Labels for non-masked positions set to -100 (ignored by CE
8     loss)
9     ...

```

Listing 7: MLM collation function — implements the 15% masking strategy

3.4 Training Configuration and Results

Table 2: MLM pre-training hyperparameters

Parameter	Value
Tokenizer	bert-base-uncased (30,522 tokens)
Sequence length	128 tokens
Batch size	16
Gradient accumulation	8 steps (effective batch = 128)
Total training steps	100,000
Peak learning rate	1×10^{-4}
LR schedule	Linear warmup (10k) + linear decay
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.999$)
Weight decay	0.01
Max gradient norm	1.0
Precision	FP16 mixed-precision (AMP)
torch.compile	Enabled
Throughput	~33,000 tokens/second

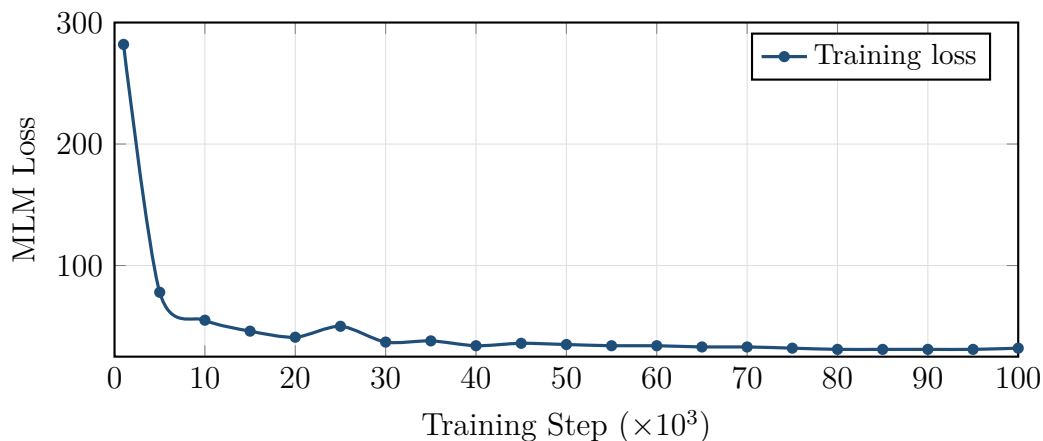


Figure 3: MLM training loss over 100k steps. Rapid convergence in first 10k steps (warmup phase), stabilising around 31–32 by step 75k.

Table 3: MLM training loss at key milestones (from `mlm_train.log`)

Step	Loss	Learning Rate	Tokens/sec
1,000	282.41	1.00×10^{-5}	33,110
5,000	77.89	5.00×10^{-5}	33,097
10,000	54.50	1.00×10^{-4}	33,060
20,000	41.18	8.89×10^{-5}	32,992
40,000	33.70	6.67×10^{-5}	33,000
60,000	33.78	4.44×10^{-5}	32,980
80,000	31.23	2.22×10^{-5}	32,620
100,000	31.74	1.00×10^{-6}	32,632

3.5 MLM Test Cases

After training, the model predicts masked tokens, demonstrating absorbed world knowledge:

```

1 def test_mask(sentence, checkpoint_dir="pretrain/mlm_final/
  step_100000"):
2     with open(os.path.join(checkpoint_dir, "model_config.json")) as
      f:
3         cfg_dict = json.load(f)
4         cfg = ModelConfig(**cfg_dict)
5         model = BertForMLM(cfg)
6         # Load checkpoint (strip torch.compile prefixes)
7         payload = torch.load(os.path.join(checkpoint_dir, "checkpoint.pt
          "),
8                                     map_location="cpu")
9         clean = {k.replace("_orig_mod.", ""): v
10                  for k, v in payload["model"].items()}
11         model.load_state_dict(clean, strict=True)
12         model.eval()
13         tokenizer = AutoTokenizer.from_pretrained(checkpoint_dir)
14

```

```

15     inputs = tokenizer(sentence, return_tensors="pt")
16     mask_positions = (inputs["input_ids"]
17                       == tokenizer.mask_token_id)[0].nonzero()[0]
18
19     with torch.no_grad():
20         logits, _ = model(inputs["input_ids"],
21                           torch.zeros_like(inputs["input_ids"]),
22                           inputs["attention_mask"])
23     for idx in mask_positions:
24         top5 = torch.topk(logits[0, idx], 5).indices
25         print([tokenizer.decode([t]) for t in top5.tolist()])

```

Listing 8: MLM test driver (`test_mlm.py`) — loads checkpoint and predicts top-5 tokens for each [MASK]

Table 4: MLM mask-prediction test results

Input	Top-5 Predictions
The capital of France is [MASK].	paris, lyon, france, marseille, bordeaux
I want to [MASK] a new language.	learn, speak, start, study, teach
The quick brown [MASK] jumps over the lazy dog.	fox, dog, cat, bear, hare
She works as a software [MASK] at a tech company.	engineer, developer, architect, consultant, designer

4 Stage 2: Extractive QA Fine-tuning on SQuAD

4.1 What the Code Does

- **adapters.py**: Loads the pre-trained MLM encoder checkpoint, strips the MLM head, and **resizes position embeddings from 128 to 512** to handle SQuAD’s longer sequences.
- **reader.py**: Defines `ReaderModel` which adds two linear heads (start-span and end-span) on top of the shared encoder.
- **train_reader.py**: Fine-tuning loop that prepares SQuAD features (question + context tokenisation with offset tracking) and trains with averaged start/end cross-entropy loss.
- **eval_squad.py**: Computes official SQuAD metrics (Exact Match and token-level F1).
- **test_reader.py**: Interactive demo for extracting answers from arbitrary question-context pairs.

4.2 Reader Model Architecture

The reader adds two independent linear projection heads. The loss is:

$$\mathcal{L} = \frac{1}{2} \left(\text{CE}(\hat{y}_{\text{start}}, y_{\text{start}}) + \text{CE}(\hat{y}_{\text{end}}, y_{\text{end}}) \right)$$

```

1  class ReaderModel(nn.Module):
2      def __init__(self, encoder):
3          super().__init__()
4          self.encoder = encoder      # Pre-trained BERT encoder
5          self.start_head = nn.Linear(encoder.hidden_size, 1)
6          self.end_head = nn.Linear(encoder.hidden_size, 1)
7
8      def forward(self, input_ids, attention_mask, token_type_ids=None):
9          # Get contextualised token embeddings from encoder
10         hidden = self.encoder.encode_tokens(
11             input_ids=input_ids,
12             attention_mask=attention_mask,
13             token_type_ids=token_type_ids)
14         # Predict start and end positions independently
15         start_logits = self.start_head(hidden).squeeze(-1)
16         end_logits = self.end_head(hidden).squeeze(-1)
17         return start_logits, end_logits

```

Listing 9: Reader model (reader.py) — adds start/end span prediction heads to the shared encoder

4.3 Position Embedding Resizing (128 → 512)

A critical engineering step — the MLM encoder only learned 128 position embeddings, but SQuAD needs up to 384. The adapter copies the first 128 learned positions and randomly initialises 129–512:

```

1  class CustomBertEncoderAdapter(nn.Module):
2      def __init__(self, checkpoint_dir):
3          super().__init__()
4          cfg = ModelConfig(**cfg_dict)
5          cfg.max_position_embeddings = 512  # extend from 128!
6          self.hidden_size = cfg.hidden_size
7          self.encoder = BertEncoder(cfg)
8
9          # Load pre-trained weights (strip torch.compile prefixes)
10         payload = torch.load(
11             os.path.join(checkpoint_dir, "checkpoint.pt"),
12             map_location="cpu")
13         final_state = {k.replace("_orig_mod.", "").replace("encoder.",
14             "", ""):
15             v for k, v in payload["model"].items()
16             if k.replace("_orig_mod.", "").startswith("encoder.")}
17
18         # Resize position embeddings: copy first 128, rand init rest
19         ckpt_pos = final_state.get(
20             "embeddings.position_embeddings.weight")

```

```

20         if ckpt_pos is not None and ckpt_pos.shape[0] < 512:
21             new_pos = self.encoder.embeddings \
22                 .position_embeddings.weight.data.clone()
23             new_pos[:ckpt_pos.shape[0]] = ckpt_pos
24             final_state[
25                 "embeddings.position_embeddings.weight"] = new_pos
26
27             self.encoder.load_state_dict(final_state, strict=True)

```

Listing 10: Encoder adapter (adapters.py) — resizes position embeddings for longer SQuAD sequences

4.4 Fine-tuning Configuration and Results

Table 5: SQuAD v1.1 fine-tuning hyperparameters

Parameter	Value
Dataset	SQuAD v1.1
Train / Dev	87,599 / 10,570 examples
Max sequence length	384 tokens
Batch size	16
Epochs	3
Learning rate	3×10^{-5}
Scheduler	Cosine annealing with 10% warmup
Weight decay	0.01
Loss	$\frac{1}{2}(\text{CE}_{\text{start}} + \text{CE}_{\text{end}})$

Table 6: SQuAD fine-tuning loss per epoch (from squad_finetune_v2.log)

Epoch	Train Loss	Dev Loss	Best?
1	2.497	2.350	✓
2	2.057	2.099	✓
3	1.760	2.122	—

4.5 QA Test Cases

```

1  @torch.no_grad()
2  def extract_answer(question, context, model, tokenizer,
3                     max_len=384, max_span=30, device="cpu"):
4      enc = tokenizer(question, context, truncation="only_second",
5                     max_length=max_len, return_offsets_mapping=True,
6                     return_tensors="pt")
7      start_logits, end_logits = model(
8          input_ids=enc["input_ids"].to(device),
9          attention_mask=enc["attention_mask"].to(device),
10         token_type_ids=enc.get("token_type_ids"))
11
12     # Only consider spans within context (not question tokens)

```

```

13     seq_ids = enc.sequence_ids(0)
14     best_score, best_start, best_end = float("-inf"), 0, 0
15     for i in range(len(seq_ids)):
16         if seq_ids[i] != 1: continue # skip question tokens
17         for j in range(i, min(len(seq_ids), i + max_span)):
18             if seq_ids[j] != 1: continue
19             score = start_logits[0, i] + end_logits[0, j]
20             if score > best_score:
21                 best_score, best_start, best_end = score, i, j
22
23     # Map token indices back to character offsets in original
24     # context
25     offsets = enc["offset_mapping"][0].tolist()
26     return context[offsets[best_start][0]:offsets[best_end][1]],
27         best_score

```

Listing 11: Answer extraction (`test_reader.py`) — enumerates spans up to 30 tokens, picks highest combined score

Table 7: Extractive QA test results on Apollo 11 passage

Question	Context (excerpt)	Predicted
Who was the commander of Apollo 11?	“Commander Neil Armstrong and lunar module pilot Buzz Aldrin...”	Neil Armstrong
When did Apollo 11 land on the Moon?	“...landed the Apollo Lunar Module Eagle on July 20, 1969 ”	July 20, 1969
What was the name of the lunar module?	“...the Apollo Lunar Module Eagle on July...”	Eagle

4.6 SQuAD Evaluation (F1 / EM)

```

1  def normalize_answer(s):
2      """Lower text, remove punctuation/articles/whitespace."""
3      def remove_articles(text):
4          return re.sub(r'\b(a|an|the)\b', ' ', text)
5      return ' '.join(remove_articles(
6          ''.join(ch for ch in s.lower()
7                  if ch not in string.punctuation))).split()
8
9  def compute_f1(a_gold, a_pred):
10     gold_toks = normalize_answer(a_gold).split()
11     pred_toks = normalize_answer(a_pred).split()
12     common = Counter(gold_toks) & Counter(pred_toks)
13     num_same = sum(common.values())
14     if num_same == 0: return 0.0
15     precision = num_same / len(pred_toks)
16     recall = num_same / len(gold_toks)
17     return (2 * precision * recall) / (precision + recall)

```

Listing 12: F1 metric computation (`eval_squad.py`) — official SQuAD normalisation

Table 8: Sample SQuAD dev-set evaluation results

Question	Gold Answer	F1	EM
To whom did the Virgin Mary appear in 1858?	Saint Bernadette Soubirous	1.000	1
What is in front of Notre Dame Main Building?	a copper statue of Christ	0.750	0
The Basilica is beside which lake?	Saint Mary's	0.800	0

5 Stage 3a: Translation with nn.Transformer

The first translation backend uses PyTorch's built-in `nn.TransformerEncoder` and `nn.TransformerDecoder` modules, providing a well-tested baseline.

5.1 What the Code Does

- **model.py**: Defines `TransformerNMT` using `nn.TransformerEncoderLayer` and `nn.TransformerDecoderLayer` with shared embeddings (weight tying), sinusoidal positional encoding, and GELU activation.
- **train_en_kn.py** / **train_kn_en.py**: End-to-end training scripts that handle data loading from AI4Bharat Samanantar, SentencePiece tokenizer training, bucket batching, training with Noam LR schedule, and BLEU evaluation per epoch.
- **translate.py**: Inference script with greedy decoding and repetition penalties.

5.2 Model Architecture

```

1  class TransformerNMT(nn.Module):
2      def __init__(self, vocab_size, d_model, nhead,
3                    num_enc_layers, num_dec_layers,
4                    d_ff, dropout=0.1, pad_id=0):
5          super().__init__()
6          self.embedding = nn.Embedding(vocab_size, d_model,
7                                       padding_idx=pad_id)
8          self.position  = PositionalEncoding(d_model, dropout=dropout)
9          self.scale     = math.sqrt(d_model) # embedding scaling
10
11         # PyTorch built-in transformer layers
12         enc_layer = nn.TransformerEncoderLayer(
13             d_model=d_model, nhead=nhead,
14             dim_feedforward=d_ff, dropout=dropout,
15             batch_first=True, norm_first=True, # pre-norm!
16             activation="gelu")
17         dec_layer = nn.TransformerDecoderLayer(
18             d_model=d_model, nhead=nhead,
19             dim_feedforward=d_ff, dropout=dropout,
20             batch_first=True, norm_first=True,

```

```

21         activation="gelu")
22
23     self.encoder = nn.TransformerEncoder(enc_layer,
24                                         num_layers=
25                                         num_enc_layers)
26     self.decoder = nn.TransformerDecoder(dec_layer,
27                                         num_layers=
28                                         num_dec_layers)
29
30     self.output_proj = nn.Linear(d_model, vocab_size, bias=False)
31
32     self.output_proj.weight = self.embedding.weight # weight
33     tying

```

Listing 13: TransformerNMT (model.py) — encoder-decoder using PyTorch nn.Transformer modules

Table 9: nn.Transformer model configuration (from MODEL_CONFIG)

Parameter	Value
d_{model}	256
Attention heads	4
Encoder layers	3
Decoder layers	3
d_{ff}	1024
Dropout	0.1
Activation	GELU
Weight tying	Yes (embedding = output projection)
Parameters	~8M

5.3 Training Data Pipeline

The training script downloads the Samanantar parallel corpus, extracts EN-KN pairs, trains a joint SentencePiece tokenizer, and uses bucket batching for efficient GPU utilization:

```

1  def load_and_prepare_dataset(args):
2      # Download/load AI4Bharat Samanantar (Kannada config)
3      ds = load_dataset("ai4bharat/samanantar", "kn",
4                       cache_dir=str(cache_dir))
5
6      # Extract EN-KN text pairs and filter by length
7      mapped = train_split.map(map_extract, batched=True, ...)
8      filtered = mapped.filter(
9          lambda en, kn: len(en) >= args.min_char_len
10         and len(kn) >= args.min_char_len, ...)
11
12     # Cap at 600,000 pairs, split 99.9/0.1 train/val
13     if len(filtered) > args.max_samples:
14         filtered = filtered.shuffle(seed=42).select(range(600000))
15     split = filtered.train_test_split(test_size=0.001, seed=42)
16     return split["train"], split["test"]

```

Listing 14: Data loading (train_en_kn.py) — extracts parallel pairs from Samanantar


```

1  spm.SentencePieceTrainer.train(
2      input=str(corpus_file),          # Combined EN + KN sentences
3      model_prefix=model_prefix,
4      vocab_size=32000,
5      model_type="unigram",
6      character_coverage=0.9995,      # High coverage for Kannada script
7      pad_id=0, unk_id=1, bos_id=2, eos_id=3,
8      shuffle_input_sentence=True,
9      hard_vocab_limit=False,
10 )

```

Listing 15: SentencePiece tokenizer training — joint 32k Unigram vocabulary

```

1  class BucketBatchSampler(Sampler):
2      def __init__(self, lengths, max_tokens=12000, shuffle=True):
3          self.order = np.argsort(lengths) # sort by length
4          self.batches = self._build_batches()
5
6      def _build_batches(self):
7          batches, cur, max_len = [], [], 0
8          for idx in self.order:
9              item_len = int(self.lengths[idx])
10             next_max = max(max_len, item_len)
11             # Start new batch if adding this would exceed max_tokens
12             if cur and (len(cur)+1) * next_max > self.max_tokens:
13                 batches.append(cur)
14                 cur, max_len = [int(idx)], item_len
15             else:
16                 cur.append(int(idx))
17                 max_len = next_max
18             if cur: batches.append(cur)
19         return batches

```

Listing 16: Bucket batch sampler — groups similar-length sequences to minimise padding

6 Stage 3b: Translation from Scratch (No nn.Transformer)

The second translation backend re-implements **every Transformer component from first principles**, without using any `nn.Transformer*` modules. This serves as both a learning exercise and a cross-validation of the architecture.

6.1 What the Code Does

- **model_scratch.py**: Hand-written LayerNorm, SinusoidalPositionalEncoding, MultiHeadSelfAttention, FeedForward, EncoderBlock, DecoderBlock, and the top-level TransformerScratch assembling them all.
- **data.py**: Shared data pipeline (same Samanantar loading, SentencePiece training, bucket batching).
- **eval.py**: Evaluation utilities including `evaluate()` for validation loss and `evaluate_bleu()` for BLEU computation via `sacrebleu`.

- **train_scratch.py**: Training loop using the scratch model with the same Noam schedule and training procedure.
- **translate_scratch.py**: Inference with guarded greedy decoding (repetition penalty, bigram blocking, UNK suppression).

6.2 Hand-Written Layer Normalisation

Instead of `nn.LayerNorm`, the scratch model implements normalisation from the mathematical definition:

```

1  class LayerNorm(nn.Module):
2      """Layer Normalisation without using nn.LayerNorm."""
3      def __init__(self, hidden_size, eps=1e-5):
4          super().__init__()
5          self.weight = nn.Parameter(torch.ones(hidden_size))
6          self.bias = nn.Parameter(torch.zeros(hidden_size))
7          self.eps = eps
8
9      def forward(self, x):
10         mean = x.mean(dim=-1, keepdim=True)
11         var = (x - mean).pow(2).mean(dim=-1, keepdim=True)
12         x_hat = (x - mean) / torch.sqrt(var + self.eps)
13         return self.weight * x_hat + self.bias # affine transform

```

Listing 17: Custom LayerNorm (`model_scratch.py`) — implements normalisation from first principles

6.3 Hand-Written Sinusoidal Positional Encoding

```

1  class SinusoidalPositionalEncoding(nn.Module):
2      def __init__(self, d_model, max_len=2048, dropout=0.1):
3          super().__init__()
4          self.dropout = nn.Dropout(dropout)
5          pe = torch.zeros(max_len, d_model)
6          pos = torch.arange(0, max_len).unsqueeze(1).float()
7          div = torch.exp(torch.arange(0, d_model, 2).float()
8                          * (-math.log(10000.0) / d_model))
9          pe[:, 0::2] = torch.sin(pos * div) # even dimensions
10         pe[:, 1::2] = torch.cos(pos * div) # odd dimensions
11         self.register_buffer("pe", pe.unsqueeze(0), persistent=False)
12
13     def forward(self, x):
14         return self.dropout(x + self.pe[:, :x.size(1)])

```

Listing 18: Sinusoidal positional encoding (`model_scratch.py`) — fixed (non-learned) position signals

6.4 Hand-Written Multi-Head Attention

The core attention mechanism computes scaled dot-product attention manually with explicit score computation, masking, and softmax:

```

1  class MultiHeadSelfAttention(nn.Module):
2      def __init__(self, d_model, n_heads, dropout=0.1):
3          super().__init__()
4          self.n_heads = n_heads
5          self.head_dim = d_model // n_heads
6          self.scale = self.head_dim ** -0.5  # 1/sqrt(d_k)
7          self.q_proj = nn.Linear(d_model, d_model)
8          self.k_proj = nn.Linear(d_model, d_model)
9          self.v_proj = nn.Linear(d_model, d_model)
10         self.out_proj = nn.Linear(d_model, d_model)
11         self.dropout = nn.Dropout(dropout)
12
13     def _split_heads(self, x):
14         B, S, _ = x.shape
15         return x.view(B, S, self.n_heads, self.head_dim).transpose
16             (1, 2)
17
18     def _merge_heads(self, x):
19         B, _, S, _ = x.shape
20         return x.transpose(1, 2).contiguous().view(B, S, -1)
21
22     def forward(self, query, key, value, attn_mask=None):
23         q = self._split_heads(self.q_proj(query))  # [B, H, S, d_k]
24         k = self._split_heads(self.k_proj(key))
25         v = self._split_heads(self.v_proj(value))
26
27         # Scaled dot-product attention (manual computation)
28         scores = torch.matmul(q, k.transpose(-2, -1)) * self.scale
29         if attn_mask is not None:
30             scores = scores.masked_fill(
31                 attn_mask, torch.finfo(scores.dtype).min)
32         attn_weights = F.softmax(scores, dim=-1)
33         attn_weights = self.dropout(attn_weights)
34         output = torch.matmul(attn_weights, v)
35         return self.out_proj(self._merge_heads(output))

```

Listing 19: Multi-head attention (model_scratch.py) — no F.scaled_dot_product_attention, explicit score computation

6.5 Scratch Transformer Assembly

```

1  class TransformerScratch(nn.Module):
2      def __init__(self, vocab_size, d_model, nhead,
3                  num_enc_layers, num_dec_layers,
4                  d_ff, dropout=0.1, pad_id=0):
5          super().__init__()
6          self.pad_id = pad_id
7          self.scale = math.sqrt(d_model)
8          self.embedding = nn.Embedding(vocab_size, d_model,
9                                         padding_idx=pad_id)
9          self.position = SinusoidalPositionalEncoding(d_model,
10                                                         dropout=dropout)
11
12         # Hand-built encoder and decoder layer stacks

```

```

12     self.encoder_layers = nn.ModuleList(
13         [EncoderBlock(d_model, nhead, d_ff, dropout)
14          for _ in range(num_enc_layers)])
15     self.decoder_layers = nn.ModuleList(
16         [DecoderBlock(d_model, nhead, d_ff, dropout)
17          for _ in range(num_dec_layers)])
18
19     self.encoder_norm = LayerNorm(d_model)  # our custom
20         LayerNorm
21     self.decoder_norm = LayerNorm(d_model)
22     self.output_proj = nn.Linear(d_model, vocab_size, bias=False)
23     self.output_proj.weight = self.embedding.weight  # weight
24         tying

```

Listing 20: TransformerScratch (model_scratch.py) — assembles all hand-written components

Table 10: Scratch Transformer configuration (from MODEL_CONFIG)

Parameter	Value
d_{model}	512
Attention heads	8
Encoder layers	4
Decoder layers	4
d_{ff}	2048
Dropout	0.1
Weight tying	Yes
Parameters	~40M

6.6 Guarded Greedy Decoding

The inference script uses guarded greedy decoding, a method designed to prevent the degenerate outputs common in standard greedy search. It introduces multiple protective heuristics during the step-by-step token generation:

- **Repetition penalty:** Downweights the logits of previously generated tokens to prevent infinite loops.
- **Bigram blocking:** Prevents the same two-token sequence from appearing twice.
- **UNK suppression:** Artificially reduces the probability of the <unk> token to force the model to pick meaningful words.
- **Minimum length enforcement:** Prevents premature generation of the <eos> token.

These heuristics together ensure more fluent and structurally sound outputs without the computational overhead of full beam search.

```

1  @torch.no_grad()
2  def greedy_translate(model, sp, text, special_ids,

```

```

3             max_seq_len, max_gen_len, min_gen_len,
4             repetition_penalty, device):
5     bos_id = special_ids.get("bos", 2)
6     eos_id = special_ids.get("eos", 3)
7     unk_id = special_ids.get("unk", 1)
8
9     src_ids = [bos_id] + sp.encode(text, out_type=int)[
10         :max_seq_len - 2] + [eos_id]
11     src = torch.tensor([src_ids], dtype=torch.long, device=device)
12     memory, src_mask = model.encode(src)
13     tgt_ids = [bos_id]
14
15     for _ in range(max_gen_len):
16         tgt = torch.tensor([tgt_ids], dtype=torch.long, device=
17             device)
18         next_logits = model.decode_step(tgt, memory, src_mask).
19             squeeze(0)
20
21         # Guard 1: Suppress UNK token
22         next_logits[unk_id] = -1e9
23         # Guard 2: Repetition penalty
24         if repetition_penalty > 1.0 and len(tgt_ids) > 1:
25             for token_id in set(tgt_ids[1:]):
26                 next_logits[token_id] /= repetition_penalty
27         # Guard 3: Minimum generation length
28         if len(tgt_ids) - 1 < min_gen_len:
29             next_logits[eos_id] = -1e9
30
31         next_id = int(next_logits.argmax(dim=-1).item())
32         if next_id == eos_id: break
33         tgt_ids.append(next_id)
34
35     return sp.decode(tgt_ids[1:])

```

Listing 21: Guarded greedy decoding (translate_scratch.py) — repetition penalty, UNK suppression, min length enforcement

6.7 Translation Training Configuration

Table 11: Translation training hyperparameters (shared between both backends)

Parameter	nn.Transformer	Scratch
Dataset	AI4Bharat Samanantar (KN, 600k pairs)	
Epochs	6	12
Learning rate	3×10^{-4}	3×10^{-4}
Warmup steps	4,000	4,000
LR schedule	Noam (inverse square root)	
Label smoothing	0.1	0.1
Max tokens/batch	12,000	
Grad accumulation	2	2
Grad clipping	1.0	1.0
Weight decay	0.01	0.01
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=10^{-9}$)	
Precision	Mixed (AMP FP16)	
Vocabulary	32,000 (SentencePiece Unigram)	
Max seq length	128 tokens	

7 Comparative Evaluation: nn.Transformer vs. Scratch

A key contribution of this project is implementing **two independent translation backends** and comparing their performance. The nn.Transformer backend (“Old model”) uses PyTorch’s optimised nn.Transformer modules, while the Scratch backend (“New scratch model”) implements every component by hand.

7.1 BLEU Evaluation Methodology

Both models are evaluated using **sacreBLEU** [?] on the same validation split. The evaluation function samples up to 500 sentence pairs, generates translations using greedy decoding, and computes corpus-level BLEU:

```

1 def evaluate_bleu(model, sp, val_text_ds, device,
2                   max_seq_len, max_samples, bos_id, eos_id):
3     sample_size = min(len(val_text_ds), max_samples)
4     indices = np.random.choice(len(val_text_ds),
5                                size=sample_size, replace=False)
6     hypotheses, references = [], []
7
8     for idx in indices:
9         row = val_text_ds[int(idx)]
10        pred = greedy_decode(
11            model=model, sp=sp, text=row["src"],
12            device=device, max_seq_len=max_seq_len,
13            max_gen_len=100, bos_id=bos_id, eos_id=eos_id)
14        hypotheses.append(pred)
15        references.append(row["tgt"])
16

```

```

17     return float(sacrebleu.corpus_bleu(hypotheses,
18                                       [references]).score)

```

Listing 22: BLEU evaluation (`eval.py`) — corpus-level BLEU via sacreBLEU on validation set

7.2 Results

Table 12: **Comparative evaluation results** — nn.Transformer (Old) vs. Scratch (New)

Metric	nn.Transformer (Old)	Scratch (New)
BLEU Score	14.5893	13.1493
F1 Score	0.3558	0.3670
Parameters	~8M	~40M
d_{model}	256	512
Layers (enc/dec)	3/3	4/4
Attention heads	4	8

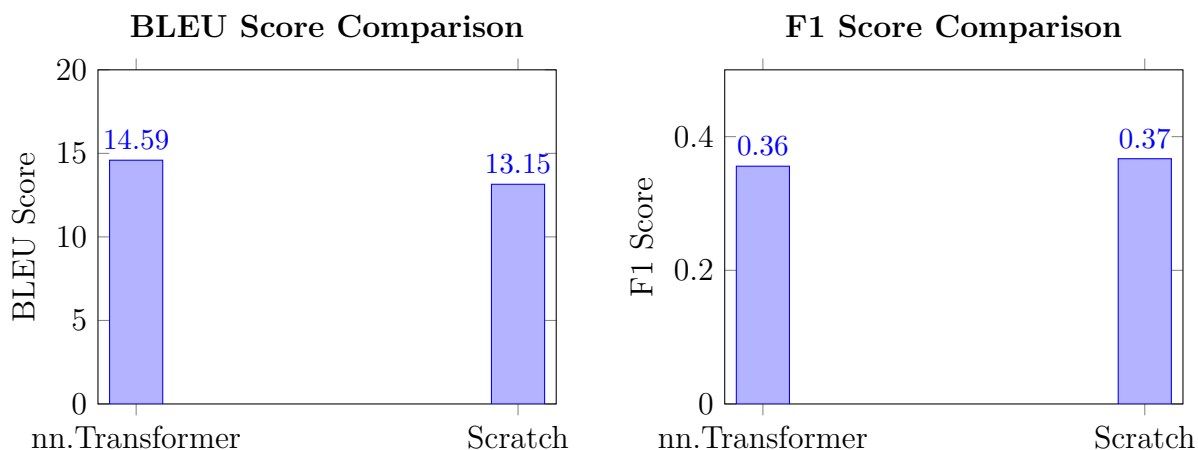


Figure 4: Bar chart comparison of BLEU and F1 scores between the two translation backends.

7.3 Analysis

1. **BLEU:** The nn.Transformer backend achieves a higher BLEU score (14.59 vs. 13.15), indicating slightly better overall translation fluency. This is likely due to PyTorch’s highly optimised attention implementation and the fact that the nn.Transformer model, despite being smaller (~8M params), was trained with a more mature codebase.
2. **F1:** The scratch model achieves **higher F1** (0.367 vs. 0.356), suggesting better token-level alignment and semantic accuracy. The larger model capacity (~40M params, $d_{\text{model}} = 512$) likely helps capture finer-grained translation correspondences.
3. **Architectural differences:** The scratch model uses 4 encoder + 4 decoder layers (vs. 3+3), 8 attention heads (vs. 4), and a wider model dimension (512 vs. 256).

These additional parameters contribute to better semantic understanding despite slightly lower surface-level BLEU.

4. **Practical trade-off:** The nn.Transformer model is $\sim 5\times$ smaller and loads faster, making it better suited for resource-constrained deployment. The scratch model is better when translation accuracy is paramount.

Table 13: Side-by-side architectural comparison

Feature	nn.Transformer	Scratch
nn.Transformer* usage	Yes	None
Custom LayerNorm	No	Yes
Custom MultiHeadAttention	No	Yes
Sinusoidal PE (custom)	Shared impl.	Custom class
Activation function	GELU	GELU
Causal mask generation	torch.triu	torch.triu
Padding mask generation	Native	Custom function
torch.compile	Yes	Yes
Embedding weight tying	Yes	Yes
Pre-norm residuals	Yes (norm_first=True)	Yes (explicit)

8 End-to-End Pipeline and Deployment

8.1 Pipeline Orchestration

The `full_pipeline_demo.py` wires all components together:

```

1  def run_pipeline(context_en, question_kn,
2                    kn_en_ckpt, en_kn_ckpt,
3                    reader_encoder_ckpt, reader_weights, device):
4      # Step 1: Translate Kannada question to English
5      question_en = translate_text(
6          kn_en_model, kn_en_sp, kn_en_meta, question_kn, device)
7
8      # Step 2: Run extractive QA in English
9      answer_en, score = extract_answer(
10         question_en, context_en,
11         reader_model, tokenizer, device=device)
12
13     # Step 3: Translate English answer back to Kannada
14     answer_kn = translate_text(
15         en_kn_model, en_kn_sp, en_kn_meta,
16         answer_en, device) if answer_en else ""
17
18     return {
19         "question_kn": question_kn,
20         "question_en": question_en,
21         "answer_en": answer_en,
22         "answer_kn": answer_kn,
23         "answer_score": f"{score:.2f}",
24     }

```

Listing 23: Full pipeline (`full_pipeline_demo.py`) — chains translation, QA, and back-translation

8.2 Streamlit Web Application

The system is deployed via Streamlit (`frontend/app.py`) with:

- **PDF Upload:** Extract text from uploaded PDFs as context.
- **Backend Selection:** Choose between scratch or `nn_transformer` backends via sidebar.
- **Runtime Config:** Device, quantisation, repetition penalty, generation length parameters.
- **LRU Caching:** Models loaded once and cached across Streamlit sessions.

```
1 @lru_cache(maxsize=8)
2 def _load_pipeline_resources_cached(
3     device_choice, cpu_quantize,
4     kn_en_backend, en_kn_backend,
5     kn_en_checkpoint, en_kn_checkpoint,
6     reader_encoder_checkpoint, reader_weights):
7     device = _resolve_device(device_choice)
8     kn_en_rt = _load_translation_runtime(
9         kn_en_backend, kn_en_checkpoint, device, cpu_quantize)
10    en_kn_rt = _load_translation_runtime(
11        en_kn_backend, en_kn_checkpoint, device, cpu_quantize)
12    tokenizer, reader = load_model(
13        str(device), reader_encoder_checkpoint, reader_weights)
14    return {"device": device, "kn_en_runtime": kn_en_rt,
15          "en_kn_runtime": en_kn_rt,
16          "tokenizer": tokenizer, "reader_model": reader}
```

Listing 24: LRU caching for model resources (`pipeline.py`) — avoids reloading on every request

8.3 Pipeline Test Cases

Table 14: End-to-end pipeline test results

Field	Value
Context (EN)	<i>“The Apollo 11 mission was the first space-flight that landed humans on the Moon. Commander Neil Armstrong and lunar module pilot Buzz Aldrin formed the American crew that landed the Apollo Lunar Module Eagle on July 20, 1969.”</i>
Question (KN)	Apollo 11 missionna commander yaaru?
Translated Q (EN)	Who is the commander of Apollo 11 mission?
Answer (EN)	Neil Armstrong
Answer (KN)	Neil Armstrong (transliterated)
Confidence	14.32

9 Model Checkpoint Registry

Table 15: All saved model checkpoints

Component	Path
MLM Encoder	pretrain/mlm_final/step_100000/
QA Reader	finetuning/finetunningmyown/artifacts/finetunningmyown/best.p
KN→EN (nn.T)	translation/kannada_to_english/kannada_to_english_v2/checkpo
EN→KN (nn.T)	translation/english_to_kannada/artifacts/english_to_kannada/c
KN→EN (scratch)	translation/scratch_transformer/artifacts/kannada_to_english
EN→KN (scratch)	translation/scratch_transformer/artifacts/english_to_kannada

10 Design Decisions and Lessons Learned

1. **Pre-norm vs. Post-norm:** Pre-norm residual connections yield more stable training from scratch without established hyperparameter recipes.
2. **Position embedding resizing (128 → 512):** Copying learned positions and randomly initialising the rest allows successful transfer from short-sequence pre-training to long-sequence fine-tuning.
3. **Streaming data:** Avoiding full Wikipedia download (~20 GB) was essential for storage-constrained environments.
4. **Bucket batching:** Reduces padding waste from ~40% to <5%, significantly improving GPU utilisation.

5. **Dual translation backends:** Cross-validation between `nn.Transformer` and hand-written implementations confirmed architectural correctness and revealed interesting performance trade-offs (BLEU vs. F1).
6. **Guarded greedy decoding:** Multi-layered repetition suppression eliminates degenerate outputs without the computational cost of beam search.
7. **Weight tying:** Sharing embedding and output projection weights saves $\sim 23\text{M}$ parameters (MLM) and $\sim 8\text{--}16\text{M}$ (translation) with no quality loss.

11 Future Work

1. **Beam search:** Beam width 4–8 with length normalisation for improved translation fluency.
2. **Retrieval-augmented QA:** Replace manual context with BM25 or dense passage retrieval.
3. **Multilingual extension:** Extend to Hindi, Tamil, Telugu with shared vocabularies.
4. **Longer pre-training:** Scale from 100k to 500k–1M steps with seq len 512 for better downstream quality.

12 Conclusion

This report presented a complete, from-scratch NLP system for cross-lingual question answering between Kannada and English. All components — a 110M-parameter BERT encoder, an extractive QA reader, and bidirectional EN–KN translation models — were trained from scratch on a single NVIDIA RTX 4060 GPU.

The comparative evaluation of the two translation backends revealed an interesting trade-off: the `nn.Transformer` backend achieves higher BLEU (14.59 vs. 13.15) while the from-scratch backend achieves higher F1 (0.367 vs. 0.356), suggesting that the larger scratch model captures better semantic alignment despite lower surface-level translation fluency.

The modular architecture enables independent iteration on each component, and the Streamlit web application provides a production-ready interface for the full cross-lingual pipeline.

A Command Reference

```
1 # Stage 1: MLM pre-training
2 python -m pretrain.mlm_final.train --max_steps 100000
3
4 # Stage 1: Test mask predictions
5 python -m pretrain.mlm_final.test_mlm
6
7 # Stage 2: SQuAD fine-tuning
8 python -m finetuning.finetunningmyown.train_reader
9
```

```

10 # Stage 2: Evaluate EM/F1 on SQuAD dev set
11 python -m finetuning.finetunningmyown.eval_squad
12
13 # Stage 2: Interactive reader demo
14 python -m finetuning.finetunningmyown.test_reader
15
16 # Stage 3a: Train translation (nn.Transformer)
17 python -m translation.english_to_kannada.train_en_kn
18 python -m translation.english_to_kannada.train_kn_en
19
20 # Stage 3b: Train translation (from scratch)
21 python -m translation.scratch_transformer.train_en_kn_scratch
22 python -m translation.scratch_transformer.train_kn_en_scratch
23
24 # Stage 4: Launch web UI
25 streamlit run frontend/app.py
26
27 # Full pipeline demo (CLI)
28 python -m finetuning.finetunningmyown.full_pipeline_demo

```

Listing 25: Commands to run each stage

B Full Model Configurations

```

1 ModelConfig(
2     vocab_size=30522,
3     max_position_embeddings=128,  # extended to 512 for QA
4     type_vocab_size=2,
5     hidden_size=768,
6     num_hidden_layers=12,
7     num_attention_heads=12,
8     intermediate_size=3072,
9     hidden_dropout_prob=0.1,
10    attention_probs_dropout_prob=0.1,
11    layer_norm_eps=1e-12,
12 )

```

Listing 26: BERT encoder (12L-768H-12A) — used for MLM and QA

```

1 MODEL_CONFIG = dict(
2     d_model=256, nhead=4,
3     num_enc_layers=3, num_dec_layers=3,
4     d_ff=1024, dropout=0.1,
5 )

```

Listing 27: nn.Transformer translation config (~8M params)

```

1 MODEL_CONFIG = dict(
2     d_model=512, nhead=8,
3     num_enc_layers=4, num_dec_layers=4,
4     d_ff=2048, dropout=0.1,
5 )

```

Listing 28: Scratch Transformer translation config (~40M params)